

## [38] Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs

---

**저자:** Y. A. Malkov and D. A. Yashunin

**학술지:** IEEE TPAMI, vol. 42, no. 4, pp. 824-836, 2018

**주제:** HNSW(Hierarchical Navigable Small World) 알고리즘 - 근사 근접 이웃 검색

### 요약

---

본 논문은 현대 벡터 검색 시스템의 핵심 알고리즘 중 하나인 HNSW(Hierarchical Navigable Small World, 계층적 네비게이팅 작은 세계 그래프)를 제시하는 매우 중요한 논문이다. HNSW는 확장 가능한 근사 근접 이웃 검색(ANN) 알고리즘으로, 높은 차원의 벡터 데이터에서 빠른 검색 속도와 높은 정확도를 동시에 달성한다.

알고리즘의 핵심은 계층적 그래프 구조로, 하단의 밀집 그래프에서 상단의 희소 그래프로 이동하면서 검색 공간을 점진적으로 좁혀나간다. 이는 "네비게이팅 작은 세계" 개념(small-world phenomenon)을 그래프에 적용한 것으로,  $O(\log n)$  시간 복잡도의 검색을 실현한다.

본 논문과의 관계: Exqutor 시스템은 HNSW를 기본 인덱싱 구조로 사용하며, 필터링 기반의 조건부 근접 이웃 검색을 HNSW 위에서 구현한다. HNSW의 계층적 구조와 네비게이팅 메커니즘을 활용하여 속성 필터를 함께 고려하는 효율적인 검색을 수행한다.

## 상세분석

---

### 38.1 문제 정의 및 동기

#### 근접 이웃 검색의 도전:

- 고차원 벡터 공간에서 정확한 KNN 검색:  $O(n)$  선형 탐색 필요
- 대규모 데이터셋(수백만~수십억): 실시간 검색 불가능
- 차원의 저주: 트리 기반 인덱스의 성능 급격히 저하
- 정확도와 속도의 트레이드오프 필요

#### HNSW의 목표:

- 근사 최근접 이웃만 검색하되 매우 빠르고 정확
- $O(\log n)$  시간 복잡도 달성
- 메모리 효율적인 인덱스 구조
- 높은 차원(1000+)에서도 안정적 성능

### 38.2 작은 세계 현상(Small-World Phenomenon)

#### 작은 세계의 특성:

- 임의의 두 노드 사이의 최단 경로가 놀랍게 짧음
- 평균 경로 길이:  $O(\log n)$  또는 더 작음
- 실제 네트워크들이 가진 특성 (소셜 네트워크, 신경망 등)

#### Watts-Strogatz 모델:

- 규칙적 격자와 무작위 네트워크의 혼합
- 몇 개의 무작위 연결로 큰 세계에서 작은 세계로 전환
- 높은 클러스터링과 짧은 경로 길이 동시 달성

#### HNSW에의 적용:

- 벡터 공간의 점들을 네트워크 노드로 생각
- 각 노드를 여러 이웃과 연결하여 작은 세계 그래프 구성
- 검색 시 몇 홉으로 근처 노드에 도달 가능

### 38.3 계층적 구조

#### 계층 개념:

- 다중 계층(multiple layers)으로 이루어진 그래프 구조
- 각 계층은 별도의 그래프로 표현
- 하단 계층: 밀집 그래프 (모든 점 포함, 많은 간선)
- 상단 계층: 희소 그래프 (적은 수의 점, 적은 간선)

#### 계층 배정:

$$L = \text{floor}(-\ln(U(0,1)) * m_l)$$

여기서:

- $U(0,1)$ : 0과 1 사이의 균등 난수
- $m_l$ : 계층 정규화 인수 (보통  $1/\ln(2)$ )
- $L$ 은 새 점의 최상단 계층 레벨

#### 계층의 확률적 특성:

- 상단 계층일수록 지수적으로 적은 점 포함
- 계층  $L$ 에 포함된 점: 약  $n * e^{(-m_l * L)}$ 개
- 균등 분포로 계층 깊이 자동 조정

### 38.4 삽입 알고리즘

#### 새로운 점 $q$ 삽입:

1. 진입점 선택
2. 최상단 계층부터 시작
3. 만약  $q$ 의  $L$ 이 기존 최상단보다 크면,  $q$ 를 새로운 진입점으로 설정

#### 4. 계층별 가까운 이웃 찾기 (Greedy Search)

```
for l = max_layer down to L(q): neighbors = greedy_search_k(q, current_nearest, m) if l > L(q): m = 1 // 상단 계층에서는 가장 가까운 1개만 유지
```

5. 상단 계층부터 하단으로 내려가며 점진적으로 검색
6. 각 계층에서 거리 기준으로 가까운  $m$ 개 이웃 선택

#### 7. 간선 추가 및 기존 간선 정리

...

for  $l = L(q)$  down to 0:

neighbors = search\_layer( $q$ , nearest,  $ef$ ,  $l$ )

$m = m$  // 하단 계층에서는  $m$ 개 이웃 유지

for 각 neighbor에:

if distance( $q$ , neighbor) < existing\_edges의최대거리:

간선 추가

else if degree(neighbor) <  $m_{max}$ :

간선 추가

else:

가장 먼 간선을 새 간선으로 교체 (휴리스틱)

...

#### 8. 프루닝(Pruning)

9. 각 노드의 차수(degree) 제한:  $m_{max} = 2 * m$
10. 새 간선이 많은 노드들은 가장 먼 간선 제거

### 38.5 검색 알고리즘

#### K-NN 검색 프로세스:

```

procedure SearchLayer(q, ep, ef, lc):
    visited = {}
    w = {} // 최종 결과 집합
    candidates = {ep}
    lowerBound = distance(q, ep)

    while candidates가 비어있지 않음:
        current = candidates에서 lowerBound가 가장 작은 점

        if distance(q, current) > lowerBound:
            break // 더 이상의 개선 불가능

        for neighbor in connections[current]:
            if neighbor not in visited:
                visited.add(neighbor)
                d = distance(q, neighbor)

                if d < lowerBound or |w| < ef:
                    candidates.add(neighbor)
                    w.add(neighbor)

                if d < lowerBound:
                    lowerBound = d

        // 결과 집합이 ef보다 크면 가장 먼 점 제거
        if |w| > ef:
            w.remove(farthest_point)
            lowerBound = max_distance_in_w

    return w

```

**ef 파라미터:**

- 검색 범위 제어 (ef: expansion factor)
- 작은 ef: 빠르지만 덜 정확
- 큰 ef: 느리지만 더 정확
- ef >= K 권장

**시간 복잡도:**

- 각 계층에서의 검색:  $O(\log n)$
- 계층 수:  $O(\log n)$
- 전체:  $O(\log^2 n)$  (이론적) 또는  $O(\log n)$  (실제 성능)

**38.6 메모리 효율성****메모리 사용량:**

```
Memory = n * (2 * m * avg_edges_per_level + data_per_point)
```

**계산:**

- 매개변수  $m = 16$ 인 경우: 약 64 바이트/포인트 (추가 인덱스)
- 벡터 데이터와 별도로 저장
- 원본 데이터의 1~2배 정도 추가 메모리

#### 압축 가능성:

- 간선 리스트 압축
- 정량화(quantization)를 통한 벡터 크기 감소
- 캐시 친화적 구조로 메모리 접근 효율성 증대

### 38.7 하이퍼파라미터 선택

#### M (최대 간선 수):

- 기본값:  $M = 16$
- 차원이 높을수록 M 증가
- 메모리와 성능의 트레이드오프

#### ef\_construction (삽입 시 확장 계수):

- 기본값: ef\_construction = 200
- 높을수록 더 좋은 그래프지만 삽입 시간 증가
- 검색 성능과 직결

#### ef (검색 시 확장 계수):

- 기본값: ef = K 또는  $K * 2$
- 동적으로 조정 가능
- 검색 속도와 정확도 조정용

### 38.8 성능 특성

#### 정확성:

- 정확한 KNN 검색 대비: 일반적으로 95% 이상 정확도
- ef 증가로 정확도 향상 가능
- M과 ef\_construction으로 사전 정확도 제어

#### 속도:

- 선형 탐색( $O(n)$ ): 백만 점 기준 수초
- HNSW( $O(\log n)$ ): 백만 점 기준 밀리초 단위
- 10~100배 속도 향상

#### 확장성:

- 수십억 개 점 지원 (메모리 허용 범위)
- 삽입/삭제도 비교적 효율적
- 배치 작업에는 최적이지 않음 (증분 삽입 가정)

### 38.9 변형 및 최적화

#### 동적 삽입/삭제:

- 기본 HNSW는 점진적 삽입 최적화
- 삭제는 비용 높음 (주변 구조 재정리 필요)
- 재구축이 더 효율적일 수 있음

#### 이질적(Heterogeneous) 그래프:

- 다른 M 값으로 다양한 밀도 제어 가능
- 계층 구조 최적화로 성능 향상

**병렬 삽입:**

- 잠금(locking) 메커니즘으로 동시 삽입 지원
- 경합(contention) 최소화 필요

### 38.10 Exqutor에서의 HNSW 활용

**기본 인덱싱:**

- Exqutor는 각 벡터를 HNSW 그래프의 노드로 저장
- 벡터의 유사도(코사인, 유클리드)에 기반한 간선 생성

**필터 통합:**

- HNSW 검색 중 필터 조건 확인
- 필터를 만족하는 노드만 결과에 포함
- 필터 조건을 통과하지 못한 노드는 경로에서 스킵

**다중 필터 처리:**

- 여러 속성 필터의 교집합 계산
- 필터된 노드 집합에서의 거리 기반 정렬
- 필터 선택도(selectivity)에 따른 검색 전략 변경

**성능 최적화:**

- 필터 조건이 매우 선택적일 때: 넓은 검색 범위 필요 (높은 ef)
- 필터 조건이 느슨할 때: 낮은 ef로도 충분
- 인덱스당 메타데이터 저장으로 필터 효율성 향상

## 추가 제기 문제

---

1. **필터 조건의 HNSW 인덱싱**: 속성 필터 조건을 HNSW 그래프 구조 내에 어떻게 효율적으로 인코딩할 수 있을까? 필터 조건별 별도의 HNSW를 유지할 때의 메모리 오버헤드는?
2. **동적 필터 변경**: 런타임 중에 필터 조건이 자주 변경될 때, HNSW 구조를 동적으로 재구성하지 않고도 효율적으로 처리할 수 있을까?
3. **필터-거리 트레이드오프의 정량화**: 필터 조건의 엄격함과 검색 거리 사이의 관계를 정량적으로 모델링하면 검색 성능을 예측할 수 있을까?
4. **계층별 필터 적용**: HNSW의 상단 계층과 하단 계층에서 필터를 다르게 적용했을 때의 성능 차이는?
5. **필터된 데이터의 클러스터링**: 같은 필터 조건을 만족하는 데이터들이 벡터 공간에서 클러스터를 형성할 때, 이를 활용한 HNSW 최적화 방법은?
6. **근사도와 필터링**: HNSW의 근사 특성(일부 최근접 이웃 누락 가능성)이 필터링과 결합될 때 최종 결과의 누락률은 얼마나 증가할까?
7. **대규모 필터 세트**: 수천 개 이상의 서로 다른 필터 조건이 있을 때, 어떤 인덱싱 전략이 가장 메모리 효율적일까?
8. **실시간 업데이트**: HNSW에 새로운 벡터를 삽입할 때 기존 필터 인덱스 구조를 어떻게 효율적으로 업데이트할 수 있을까?
9. **필터 검색 순서 최적화**: 다중 필터 조건이 있을 때, 검색 성능을 최대화하기 위한 필터 적용 순서 결정 알고리즘은?
10. **벡터 임베딩 모델과의 상호작용**: BERT, GPT 등 다양한 임베딩 모델에서 생성된 벡터들의 특성이 HNSW 구조에 어떻게 영향을 미치는가? 모델별 최적의  $M$ ,  $ef$  값은?